C Programming

8_preprocessor

Этапы сборки программы

```
main.c --[препроцессор]--> main.i --[компилятор]--> main.s
main.s --[ассемблер]--> main.o --[линковщик]--> a.out
```

- препроцессор: раскрывает #include, #define, #if (текст - текст)
- компилятор: Си - ассемблер
- ассемблер: ассемблер - объектный файл .o
- линковщик: склеивает .o и библиотеки в исполняемый файл

Флаг -E останавливает сборку сразу после препроцессора:

gcc main.c -E -o main.i

Директива #include

Заголовочный файл нужен только для:
Описания прототипов функций либо объявления новых типов данных (структуры, объединения)

```
#include <stdio.h>
```

Угловые скобки <> - говорит компилятору искать в стандартных путях (обычно /usr/include в linux).

"" - Искать в текущем каталоге

```
#include "my_header.h"
```

Опцией -I компилятору можно задать путь до заголовочных файлов компилятору, чтобы в include не прописывать полный путь до заголовочного файла.

```
gcc prog.c -o prog -I ./include/
```

#define

#define - директива, предназначенная для макро определений.
С точки зрения хорошего стиля кода принято все макро определения пишут **большими буквами.**

#define используется обычно в двух случаях:

- для определения констант (#define N 5)
- для определения макросов (подобие функции, но не это не функция!)

```
#define K 123 // K определена как 123

// ..

#undef K // а теперь K больше не существует
```

```c
#define MIN(a, b) ((a < b) ? a : b)

MIN(1, 2);//-> ((1 < 2) ? (1 : 2))

#define ADD(a, b) a + b

c = ADD(5, 3); //-> 5 + 3
```

```c
#define ADD(a, b) a + b
int a = 5, b = 3;
int c = ADD(a, b); // c?
int d = 4 * ADD(a, b); // d?
```

```
#define MIN(a, b) ((a < b) ? a : b)

MIN(func1(), func2());
```

Правила для макросов

```
#define ADD(a, b) ((a) + (b))   // 4 * ADD(a, b) = 32, а не 23
MIN(i++, 10);                   // i увеличится дважды
#define LOG(x) do { ... } while (0)
```

- скобки вокруг каждого аргумента и вокруг всего выражения
- аргумент подставляется столько раз, сколько встречается в теле
- несколько операторов - в do { ... } while (0)
- нужна функция - пишите функцию, а макрос оставьте константам

Предопределенные константы

- __LINE__ - номер строки
- __FILE__ - имя файла
- __FUNCTION__ - имя функции

```
printf("Error at LINE=%d, FILE=%s, function=%s\n",
                                __LINE__, __FILE__, __FUNCTION__);

printf("%s:%d\n", __FILE__, __LINE__); // CTRL+ЛКМ по этой ссылке в VSCode
```

- __DATE__ - дата компиляции программы
- __TIME__ - дата компиляции программы

```
printf("Compilation time: date=%s, time=%s\n", __DATE__, __TIME__);
```

Условная компиляция

```
#if B == 5
// ..
#else
// ..
#endif
```

```
gcc src/cond.c -DB=5
```

```
#ifdef DEBUG
    // ..
#endif
```

```
#ifndef DEBUG
    // ..
#else
    // ..
#endif
```

```
gcc src/cond.c -DDEBUG=1
```

Все эти директивы выполняются во время компиляции, а не во время работы программы! Т.е. директивы не будут работать с внутренними переменными языка Си.

Полезно для добавления дебаг кода, для переносимости на различные архитектуры, для учитывания версии библиотек и т.д.

```c
int sum(int x, int y) {
    int res = x + y;
#ifdef DEBUG
    // этот код будет скомпилирован только если DEBUG
определен
    printf("DEBUG: Value of x = %d\n", x);
    printf("DEBUG: Value of y = %d\n", y);
    printf("DEBUG: Sum calculated as %d\n", res);
#endif
    return res;
}
```

```c
#define PRINT_DEBUG(format, ...) \
do {
printf("%s:%d:%s(): " format, __FILE__, __LINE__, __FUNCTION__,
##__VA_ARGS__);
} while(0)
```

```
...
PRINT_DEBUG("Error: %s\n", message); // `;` точка с запятой
обязательно
...
```

do { if (1) printf("%s:%d:%s(): a !=%d\n", "cond.c", 5, "main()", a); } while(0);

Защита от повторного включения

```
// test.h

#ifndef TEST_H
#define TEST_H
// ..
int a; // определение
чего-либо
// ..
#endif // TEST_H
```

```
// test2.h

#include "test.h"
```

```
// test.c

#include "test.h"
#include "test2.h"

// ..
```

```
// современная альтернатива
#pragma once
```

Итого: препроцессор общается с компилятором

Все директивы отрабатывают до сборки. К моменту запуска программы их уже нет.

- в бинарнике нет #include, #define и #if - только результат подстановки
- #if не видит переменные: их еще не существует, есть только текст
- #define не занимает память и не имеет адреса
- код под #ifdef DEBUG в релизной сборке исчезает целиком
- странное поведение директивы - ищите ошибку в тексте, а не в логике

```
gcc main.c -E -o main.i   # посмотреть, что получилось
```

Github

https://github.com/kruffka/C-Programming

